
MATERIAL DE TREINAMENTO

APScheduler 3.x

Agendamento de tarefas em Python

3 semanas

5 módulos

Projeto incluído

Sem custos

Versão para uso interno · Python 3.8+

Sumário

Semana 1

Módulo 1 — O que é o APScheduler e como instalar

Módulo 2 — Criando os primeiros jobs

Semana 2

Módulo 3 — Os três tipos de trigger

Módulo 4 — Controlando jobs em tempo de execução

Semana 3

Módulo 5 — Projeto: Gerador de Relatórios CSV

Referência rápida

O que é o APScheduler e como instalar

Por que usar um scheduler?

Imagine que você precisa gerar um relatório todo dia às 8h, ou verificar o conteúdo de uma pasta a cada 10 minutos. Fazer isso com um loop e `time.sleep()` funciona, mas vira um problema quando você precisa de múltiplas tarefas, horários precisos ou controle sobre o que está rodando. O APScheduler resolve exatamente isso.

Os quatro conceitos centrais

A biblioteca funciona com quatro peças que trabalham juntas:

Peça	O que faz
Scheduler	É o motor. Você cria, inicia e controla por ele.
Trigger	Define quando o job vai rodar (a cada X minutos, num horário fixo...).
JobStore	Define onde os jobs ficam guardados (memória ou banco de dados).
Executor	Define como o job é executado (em thread, processo, etc.).

Para começar, você só vai precisar do Scheduler e do Trigger. JobStore e Executor têm configurações padrão que funcionam bem nos exercícios iniciais.

Instalação

Use sempre a versão 3.x neste treinamento. A versão 4 tem uma API diferente e não é compatível com os exemplos aqui.

```
pip install apscheduler==3.10.4
# Para confirmar que instalou certo:
python -c "import apscheduler; print(apscheduler.__version__)"
# Deve imprimir: 3.10.4
```

Atenção: Anote a versão no seu `requirements.txt` desde o início: `apscheduler==3.10.4`. Isso evita que uma atualização futura quebre o código.

Um ponto de Python que você precisa entender antes de continuar

O APScheduler recebe uma função como argumento — sem parênteses. Isso é diferente de chamar a função. Veja a diferença:

```
def minha_tarefa():  
    print('rodando')  
  
    # CERTO – passa a função em si (o scheduler vai chamá-la no momento certo)  
    scheduler.add_job(minha_tarefa, ...)  
  
    # ERRADO – chama a função agora e passa o resultado (None) pro scheduler  
    scheduler.add_job(minha_tarefa(), ...)
```

EXERCÍCIO 1.1

Crie um arquivo chamado **ex01.py**. Defina uma função **ola()** que imprime 'Olá, APScheduler!'. Crie uma segunda função **executar(fn)** que chama `fn()`. Chame `executar` passando `ola` sem parênteses e confirme que a saída está correta.

Criando os primeiros jobs

BlockingScheduler e BackgroundScheduler

Existem dois schedulers que você vai usar com frequência. A diferença principal é simples:

	BlockingScheduler	BackgroundScheduler
Trava o processo?	Sim	Não
Roda em thread?	Não	Sim, em segundo plano
Quando usar?	Scripts avulsos, CLIs	Dentro de uma aplicação maior

Nos exercícios desta semana você vai usar o BlockingScheduler. O BackgroundScheduler aparece no projeto da Semana 3.

Seu primeiro scheduler funcionando

```
from apscheduler.schedulers.blocking import BlockingScheduler

def tarefa():
    print('executando...')

scheduler = BlockingScheduler()
scheduler.add_job(tarefa, 'interval', seconds=5)
print('Iniciando. Use Ctrl+C para parar.')
scheduler.start()
```

Observação: Execute esse script no terminal e observe a saída. O processo fica parado aqui — esse é o comportamento do BlockingScheduler.

O método add_job() em detalhe

Esse é o método central da biblioteca. Os parâmetros mais importantes:

```
scheduler.add_job(
    func, # função a rodar (sem parênteses)
    trigger, # 'interval', 'cron' ou 'date'
    id='meu_job', # identificador único – use sempre
    args=[], # argumentos posicionais para func
    kwargs={}, # argumentos nomeados para func
    max_instances=1, # evita execuções simultâneas
    replace_existing=True, # não duplica se o ID já existir
```

```
)
```

Passando argumentos para o job

```
from apscheduler.schedulers.blocking import BlockingScheduler
def notificar(setor, prioridade):
    print(f'Notificação para {setor} | prioridade: {prioridade}')
    scheduler = BlockingScheduler()
    scheduler.add_job(
        notificar,
        'interval',
        seconds=10,
        id='notif_producao',
        kwargs={'setor': 'producao', 'prioridade': 'alta'}
    )
    scheduler.start()
```

EXERCÍCIO 1.2

Crie um script com `BlockingScheduler` que adiciona três jobs diferentes, cada um com um ID único. Todos chamam a mesma função **registrar(nome)**, que imprime o nome recebido. Configure intervalos diferentes para cada um e observe a saída.

Os três tipos de trigger

O trigger diz ao scheduler quando rodar um job. Cada um dos três tipos tem um caso de uso diferente — escolher o errado é o erro mais comum de quem está começando.

Trigger	Use quando...	Exemplo real
interval	A tarefa precisa repetir em ciclos fixos	Checar uma pasta a cada 5 min
cron	A tarefa tem horário fixo, como um alarme	Relatório todo dia às 8h
date	A tarefa roda uma única vez no futuro	Enviar lembrete em 2 horas

Trigger interval

Repete o job com um intervalo constante entre execuções. Aceita: weeks, days, hours, minutes, seconds.

```
# A cada 30 minutos
scheduler.add_job(fn, 'interval', minutes=30)

# A cada 1 hora e 30 minutos
scheduler.add_job(fn, 'interval', hours=1, minutes=30)

# Com janela de tempo (só roda entre essas datas)
scheduler.add_job(fn, 'interval', hours=1,
start_date='2024-01-15 08:00:00',
end_date='2024-12-31 18:00:00')
```

Trigger cron

Funciona como o cron do Linux: você define dia da semana, hora, minuto e o scheduler cuida do resto. Parâmetros: year, month, day, week, day_of_week, hour, minute, second.

```
# Todo dia às 08:30
scheduler.add_job(fn, 'cron', hour=8, minute=30)

# Segunda a sexta às 07:00
scheduler.add_job(fn, 'cron', day_of_week='mon-fri', hour=7, minute=0)

# Primeiro dia de cada mês à meia-noite
scheduler.add_job(fn, 'cron', day=1, hour=0, minute=0)

# A cada 15 minutos, das 8h às 18h, dias úteis
scheduler.add_job(fn, 'cron',
day_of_week='mon-fri', hour='8-18', minute='*/15')
```

Atenção: Defina sempre o fuso horário quando usar cron. Sem ele, o scheduler usa UTC e seu job vai rodar no horário errado no Brasil.

```
# Criando o scheduler com fuso horário correto
from apscheduler.schedulers.blocking import BlockingScheduler
scheduler = BlockingScheduler(timezone='America/Sao_Paulo')
```

Trigger date

Executa o job uma única vez, na data e hora que você especificar. Depois da execução, o job é removido automaticamente.

```
from datetime import datetime, timedelta
# Data fixa
scheduler.add_job(fn, 'date', run_date='2024-06-15 14:30:00',
kwargs={'msg': 'Reunião em 30 minutos'})
# Daqui a 2 horas (calculado em tempo real)
daqui_2h = datetime.now() + timedelta(hours=2)
scheduler.add_job(fn, 'date', run_date=daqui_2h)
```

EXERCÍCIO 2.1

Num único script, crie três jobs usando os três triggers:

- 1. interval — imprime 'ping' a cada 8 segundos
- 2. date — imprime 'job único!' daqui a 20 segundos
- 3. cron — imprime 'cron ok' no próximo minuto cheio (use minute=*)

Execute o script por pelo menos 90 segundos e observe quantas vezes cada um dispara. O job date deve aparecer exatamente uma vez.

Controlando jobs em tempo de execução

Em situações reais você vai precisar pausar, reativar ou remover jobs sem reiniciar o programa. Essa seção cobre os métodos para fazer isso.

Métodos de controle

```
from apscheduler.schedulers.background import BackgroundScheduler
import time

def tarefa(): print('rodando')

scheduler = BackgroundScheduler()
scheduler.add_job(tarefa, 'interval', seconds=5, id='job_a')
scheduler.start()

time.sleep(12)
scheduler.pause_job('job_a') # para de executar, mas continua registrado
print('pausado')

time.sleep(8)
scheduler.resume_job('job_a') # volta a executar normalmente
print('reativado')

time.sleep(8)
scheduler.remove_job('job_a') # remove permanentemente
print('removido')

time.sleep(3)
scheduler.shutdown()
```

Inspecionando e alterando jobs

```
# Listar todos os jobs ativos
for job in scheduler.get_jobs():
    print(job.id, '|', job.next_run_time)

# Buscar um job específico (retorna None se não existir)
job = scheduler.get_job('job_a')
if job:
    print('próxima execução:', job.next_run_time)

# Mudar o intervalo de um job sem removê-lo
scheduler.reschedule_job('job_a', trigger='interval', seconds=30)
```

Lidando com falhas: misfire e coalesce

Se o programa ficar offline por algum tempo, alguns jobs podem perder o horário. Dois parâmetros controlam o que acontece nesses casos:

```
scheduler.add_job(
    minha_tarefa,
    'interval',
    minutes=5,
    id='tarefa_importante',
    misfire_grace_time=60, # se atrasou até 60s, ainda executa
    coalesce=True, # se perdeu vários ciclos, executa só uma vez
    max_instances=1, # não roda em paralelo consigo mesmo
)
```

Ouvindo eventos do scheduler

Você pode registrar uma função que é chamada toda vez que um job termina, falha ou perde o horário. Isso é o lugar certo para logging e alertas.

```
from apscheduler.events import EVENT_JOB_EXECUTED, EVENT_JOB_ERROR

def ao_finalizar(event):
    if event.exception:
        print(f'[ERRO] {event.job_id} falhou: {event.exception}')
    else:
        print(f'[OK] {event.job_id} concluído')
    scheduler.add_listener(ao_finalizar, EVENT_JOB_EXECUTED | EVENT_JOB_ERROR)
```

EXERCÍCIO 2.2

Crie um script com BackgroundScheduler que adiciona um job com interval de 3 segundos. Usando um listener e uma variável de contagem no escopo externo, faça o job se remover automaticamente após 5 execuções. O programa deve encerrar logo depois.

Projeto: Gerador de Relatórios CSV

Este projeto consolida tudo que foi visto nas duas primeiras semanas. Você vai construir um programa que monitora uma pasta, lê arquivos CSV de vendas e gera resumos automáticos em horários configuráveis — sem nenhuma dependência externa além do APScheduler.

O que o projeto faz

- Lê arquivos CSV de uma pasta de entrada a cada intervalo configurado
- Calcula o total de vendas por categoria
- Salva um arquivo de resumo na pasta de saída com timestamp no nome
- Registra em log todas as operações e eventuais erros
- Sobrevive a reinicializações (jobs persistidos em SQLite)

Estrutura de arquivos

```
projeto_relatorios/  
main.py # ponto de entrada – inicia o scheduler  
processador.py # lógica de leitura e geração de relatório  
dados/  
  entrada/ # coloque os CSVs aqui  
  saida/ # relatórios gerados aparecem aqui  
scheduler.log # log de execuções  
jobs.db # jobs persistidos (criado automaticamente)
```

Arquivo de exemplo: dados/entrada/vendas.csv

```
data,produto,categoria,quantidade,valor_unitario  
2024-06-01,Notebook,Informatica,2,3500.00  
2024-06-01,Mouse,Informatica,10,89.90  
2024-06-02,Cadeira,Escritorio,3,650.00  
2024-06-02,Monitor,Informatica,1,1200.00  
2024-06-03,Mesa,Escritorio,2,480.00  
2024-06-03,Teclado,Informatica,5,150.00
```

processador.py

```
import csv  
import os
```

```

import logging
from datetime import datetime
from collections import defaultdict

logger = logging.getLogger('relatorios')
PASTA_ENTRADA = 'dados/entrada'
PASTA_SAIDA = 'dados/saida'

def gerar_relatorio():
    """Lê todos os CSVs da pasta de entrada e gera um resumo por categoria."""
    logger.info('Iniciando geração de relatório...')
    totais = defaultdict(float)
    arquivos_lidos = 0

    for nome in os.listdir(PASTA_ENTRADA):
        if not nome.endswith('.csv'):
            continue

        caminho = os.path.join(PASTA_ENTRADA, nome)
        try:
            with open(caminho, newline='', encoding='utf-8') as f:
                leitor = csv.DictReader(f)
                for linha in leitor:
                    cat = linha['categoria']
                    valor = float(linha['valor_unitario'])
                    qtd = int(linha['quantidade'])
                    totais[cat] += valor * qtd
                    arquivos_lidos += 1
        except Exception as e:
            logger.error(f'Erro ao ler {nome}: {e}')
            continue

    if not totais:
        logger.warning('Nenhum dado encontrado. Relatório não gerado.')
    return

    # Salva o relatório
    os.makedirs(PASTA_SAIDA, exist_ok=True)
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    saida = os.path.join(PASTA_SAIDA, f'resumo_{timestamp}.csv')
    with open(saida, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['categoria', 'total_vendas'])
        for cat, total in sorted(totais.items()):
            writer.writerow([cat, f'{total:.2f}'])
    logger.info(f'Relatório gerado: {saida} ({arquivos_lidos} arquivo(s) processado(s))')

```

main.py

[illegible]

- 4. Crie um arquivo `config.py` com as configurações (intervalo, pastas, timezone) e importe-o no `main.py` — evite números mágicos no código

Referência rápida

Métodos do scheduler

Método	O que faz
<code>scheduler.start()</code>	Inicia o scheduler
<code>scheduler.shutdown(wait=True)</code>	Encerra aguardando jobs em andamento
<code>scheduler.add_job(fn, trigger, ...)</code>	Adiciona um job
<code>scheduler.remove_job(id)</code>	Remove um job pelo ID
<code>scheduler.pause_job(id)</code>	Pausa (não remove)
<code>scheduler.resume_job(id)</code>	Reativa um job pausado
<code>scheduler.reschedule_job(id, ...)</code>	Muda o trigger de um job
<code>scheduler.get_job(id)</code>	Retorna o job ou None
<code>scheduler.get_jobs()</code>	Lista todos os jobs ativos
<code>scheduler.add_listener(fn, mask)</code>	Registra um listener de eventos

Expressões cron úteis

Parâmetros	Quando dispara
<code>hour=8, minute=0</code>	Todo dia às 08:00
<code>day_of_week='mon-fri', hour=9</code>	Dias úteis às 09:00
<code>day=1, hour=0, minute=0</code>	Primeiro dia de cada mês, meia-noite
<code>minute='*/15'</code>	A cada 15 minutos
<code>hour='8-18', minute='*/30'</code>	A cada 30 min, entre 8h e 18h
<code>day_of_week='mon', hour=9, minute=0</code>	Toda segunda-feira às 09:00

Eventos disponíveis para listeners

Constante	Quando é chamada
<code>EVENT_JOB_EXECUTED</code>	Job terminou sem exceção
<code>EVENT_JOB_ERROR</code>	Job lançou uma exceção

EVENT_JOB_MISSED	Job perdeu o horário de execução
EVENT_JOB_ADDED	Um job foi adicionado
EVENT_JOB_REMOVED	Um job foi removido
EVENT_SCHEDULER_STARTED	Scheduler foi iniciado
EVENT_SCHEDULER_SHUTDOWN	Scheduler foi encerrado

O que não esquecer

- Sempre passe a função sem parênteses: `add_job(minha_funcao, ...)` e não `add_job(minha_funcao(), ...)`
- Use IDs explícitos em todos os jobs — facilita debug e evita duplicatas
- Defina timezone no scheduler quando usar cron, especialmente em servidores
- Use `replace_existing=True` se o job for re-registrado ao reiniciar com jobstore persistido
- `shutdown(wait=True)` garante que jobs em andamento terminem antes de encerrar
- Em produção, registre sempre um listener de erro — falhas silenciosas são difíceis de rastrear